

Making and keeping duplicate copies of a database on other servers is known as database replication. It is essential for improving modern systems' scalability, reliability, and data availability.

- By distributing their data across multiple servers, organizations can guarantee that it will remain accessible even in the case of a server failure.
- This redundancy also improves data reliability because many copies are available to recover data in the case of corruption or loss.
- Database replication can help in workload distribution among servers, boosting scalability and performance.

Importance of Database Replication

Database replication is important for several reasons:

- **High Availability:** Data availability is guaranteed by replication, even in the event that one or more servers fail. Applications can continue to run uninterrupted by keeping copies of their data on several servers.
- **Disaster Recovery:** In the case of a disaster, replication offers a way to restore data. After a disaster, businesses can quickly resume operations by keeping copies of their data in many locations.
- **Load Balancing:** It allows for distributing read queries across multiple servers, reducing the load on any single server and improving performance.
- **Fault Tolerance:** It improves fault tolerance by ensuring that if one server fails, another can take over with minimal disruption.
- **Scalability:** It can improve scalability by allowing for the distribution of write operations across multiple servers, reducing the load on any single server.
- **Data Locality:** It can be used to bring data closer to users, reducing latency and improving the user experience.

How does Database Replication works?

Here are the steps explaining how database replication works:

- **Step 1: Identify the Primary Database (Source):** A primary (or master) database is chosen as the main source of truth where data changes originate.
- **Step 2: Set Up Replica Databases (Targets):** One or more replicas (or secondary databases) are configured to receive data from the primary database.
- **Step 3: Data Changes Captured:** Any updates, inserts, or deletes in the primary database are recorded, typically through a transaction log or change data capture mechanism.
- **Step 4: Transmit Changes to Replicas:** The captured changes are sent to replica databases over the network in real-time or at scheduled intervals.
- **Step 5: Apply Changes on Replicas:** The replicas apply these updates to keep their data in sync with the primary database.
- **Step 6: Monitor and Maintain Synchronization:** The system ensures replicas stay up-to-date and handles issues like delays or conflicts during synchronization.
- **Step 7: Read or Write Operations:** Applications can read data from replicas (to reduce load on the primary) and may write to the primary, depending on the replication model (e.g., master-slave or multi-master).

on the primary) and may write to the primary, depending on the replication model (e.g., Master-Slave, Master-Master).

Types of Database Replication

Let's understand the different types of database replication:

1. Master-Slave Replication:

- The process of copying and synchronizing data from a primary database (the master) to one or more secondary databases (the slaves) is known as master-slave replication.
- In this configuration, all write operations, including inserts, updates, and deletions, must be received by the master database.
- The slave databases keep a duplicate of the data and replicate the modifications made to the master database.

2. Master-Master Replication/Multi-Master Replication:

- Master-master replication, also known as bidirectional replication, is a setup in which two or more databases are configured as master databases, and each master can accept write operations.
- This means that changes made to any master database are replicated to all other master databases in the configuration.

3. Snapshot Replication:

- Creating a copy of the whole database at a certain moment in time and then replicating that snapshot to one or more destination servers is known as snapshot replication.

4. Transactional Replication:

- One way to maintain several copies of a database synchronized in real-time is through transactional replication.
- This means any modifications made to a particular table (or group of tables) in one database—referred to as the publisher—are instantly copied to other databases—referred to as subscribers.

5. Merge Replication:

- Merge replication is a database synchronization method allowing both the central server (publisher) and its connected devices (subscribers) to make changes to the data, resolving conflicts when necessary.

Strategies for Database Replication

Database replication strategies determine how to select data, copy and distribute it between databases to gain specific goals such as scalability, availability, and efficiency. Some common database replication strategies include the following:

- **Full Replication:** Also referred to as full database replication, this is a technique in which the whole database is replicated to one or more destination servers. All the tables, rows, and columns in the database are copied to the destination servers. The replicas thus obtain an exact copy of the original database.
- **Partial Replication:** This method involves not replicating the entire database, but merely a subset of it, such as particular tables, rows, or columns. This method can be useful when only

subset of it, such as particular tables, rows, or columns. This method can be useful when only specific data has to be reproduced for reporting, analysis, or other reasons, and it enables a more effective use of resources.

- **Selective Replication:** It is a database replication strategy that involves replicating data based on predefined criteria or conditions. Unlike full replication, which replicates the entire database, or partial replication, which replicates a subset of the database, selective replication allows for more granular control over which data is replicated.
- **Sharding:** It is a database scaling technique that involves partitioning data across multiple database instances (shards) based on a key. This approach allows for distributing the workload and data storage across multiple servers, improving scalability and performance.
- **Hybrid Replication:** It is a database replication strategy that combines multiple replication techniques to achieve specific goals. This approach allows for the customization of replication methods based on the requirements of different parts of the database or application.

Configurations of Database Replication in System Design

To accomplish particular objectives related to data consistency, availability, and performance, database replication can be set up and run in a variety of ways:

1. Synchronous Replication Configuration:

- It is a database replication technique that replicates data changes in real-time to one or more replicas. Until at least one copy acknowledges receiving the changes, the transaction isn't considered committed.
- This technique offers a high degree of data consistency by guaranteeing that the main database and replicas of it are constantly in sync.

2. Asynchronous Replication Configuration:

- Data changes performed on the primary database are replicated to one or more replicas using this database replication technique, which does not wait for the clones to acknowledge them.
- Faster transaction processing on the primary database is possible with this approach, but there may be a small lag in data consistency between the primary and replica(s).

3. Semi-synchronous Replication Configuration

- A database replication technique called semi-synchronous replication combines elements of synchronous and asynchronous replication.
- While other copies are updated asynchronously for improved efficiency, semi-synchronous replication ensures excellent data consistency for essential data by replicating changes to at least one replica synchronously.

Challenges with Database Replication

Some of the challenges with Database Replication are:

- **Data Consistency:** It can be difficult to maintain consistency among replicas, particularly in asynchronous replication situations where data replication may be delayed.
- **Complexity:** System complexity is increased by database replication, which requires thorough setup and administration to guarantee accurate and effective data replication.
- **Cost:** Setting up and maintaining a replicated database environment can be costly, especially

- **Cost:** Setting up and maintaining a replicated database environment can be costly, especially for large-scale deployments with multiple replicas.
- **Conflict Resolution:** When the same data is changed on multiple replicas at once in multi-master replication environments, conflicts might arise that require conflict resolution techniques.
- **Latency:** Synchronous replication, which requires acknowledgment from replicas before committing transactions, can introduce latency and impact the performance of the primary database.